Replicating SO(2)-Equivariant Reinforcement Learning for Robot Manipulation, with an Extension to ManiSkill

Wade Williams, Joseph Horwitz

Khoury College of Computer Sciences, Northeastern University
CS 5180 Reinforcement Learning, Spring 2026
{williams.wad, horwitz.jo}@northeastern.edu

Abstract

We reimplement the $SO(2)$-Equivariant SAC and DQN algorithms of Wang, Walters, and Platt (2022) from scratch and reproduce their main close-loop manipulation results on three BulletArm tasks: block pulling, object picking (block picking for the SAC comparison), and drawer opening. Each equivariant agent is compared against the same data-augmentation and contrastive baselines used in the paper (DrQ, RAD, FERM, CURL) across two random seeds. Equivariant SAC reaches a discounted evaluation reward of ≈ 0.93 on all three tasks; every non-equivariant SAC baseline stays near zero across the full 20,000-step budget, except for DrQ on block pulling, which partially learns. Equivariant DQN leads on block pulling and object picking, and the equivariance advantage is smaller for DQN than for SAC, matching the trend reported in the paper. We then extend the SAC comparison to ManiSkill 3 on PickCube-v1, the closest ManiSkill task to the paper's object-picking setup. Under a longer 100,000-step training run with four seeds, Equivariant SAC achieves higher mean returns than the CNN SAC baseline on ManiSkill, matching the direction of the paper's BulletArm result. A contrastive-pretraining pass for the FERM baseline and an extension to the two remaining paper tasks are natural next steps. Our code, plots, and replication protocol are released with this report.

Introduction

Sample efficiency is a central problem in reinforcement learning: a policy is only useful if it can be learned from a reasonable amount of environmental interaction. This matters especially in robotics, where every sample comes from a physical robot, rollouts cost wall-clock time and mechanical wear, and an unsafe policy can damage the robot or its surroundings. One approach to improving sample efficiency is data augmentation: creating artificial experiences by applying affine transformations (e.g., image translation or rotation) to the states in a transition (Laskin et al. 2020; Kostrikov, Yarats, and Fergus 2021). These methods implicitly assume that the transition and reward dynamics are invariant under those transformations. Some approaches make this explicit by adding a contrastive loss that pushes the agent toward translation-invariant features (Laskin, Srinivas, and Abbeel 2020; Zhan et al. 2020).

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Geometric deep learning takes a more direct route to the same inductive bias. Cohen and Welling (2016a,b) showed that equivariant neural networks can represent transformation-invariant functions by construction. Both approaches target the same bias, but equivariant architectures encode it in the model rather than in the training data. With augmentation, the model has to learn equivariance alongside the task, which usually costs more training time and more model capacity. Even then, the equivariance is only approximate, while equivariant networks guarantee it exactly and often generalize better (Wang, Walters, and Yu 2020). Early work in equivariant reinforcement learning focused on discrete groups in grid-world and toy domains (van der Pol et al. 2020a,b; Mondal et al. 2020).

Wang, Walters, and Platt (2022) scaled the approach to realistic visuomotor manipulation. They showed that $SO(2)$-equivariant networks achieve substantially better sample efficiency than the standard and data-augmentation baselines on close-loop PyBullet tasks. Their result has been a reference point for later work that extends architectural equivariance to new policy classes and larger symmetry groups, including Equivariant Diffusion Policy (Wang et al. 2024), Spherical Diffusion Policy (Zhu et al. 2025), and 3D equivariant visuomotor policies on point clouds (Hu et al. 2025). In this project, we reproduce the main results of Wang, Walters, and Platt (2022): Equivariant SAC and Equivariant DQN against the same data-augmentation and contrastive baselines Wang et al. used, on three close-loop PyBullet manipulation tasks (block pulling, block picking, and drawer opening). We then extend the comparison to ManiSkill 3, a GPU-parallel manipulation simulator, to see whether the sample-efficiency gap carries over to a different physics backend and camera pipeline.

Related Work

Data Augmentation and Contrastive Learning

Most work on improving sample efficiency in visuomotor reinforcement learning uses some form of pixel-space data augmentation during training. DrQ (Kostrikov, Yarats, and Fergus 2021) augments each minibatch with random shifts and averages the Q-target across the augmented copies. RAD (Laskin et al. 2020) applies random crops directly to observations as they enter the actor and critic, with no con-

sistency loss. Both methods train the encoder to be robust to affine transformations through repeated exposure to augmented samples. A related line of work adds a contrastive objective on top of the augmentation (van den Oord, Li, and Vinyals 2018). CURL (Laskin, Srinivas, and Abbeel 2020) trains the encoder to pull augmented views of the same observation together in latent space, and FERM (Zhan et al. 2020) adapts that pipeline to sample-efficient robotic manipulation by combining SAC, random crop, and contrastive pretraining. Wang et al. compare against these four methods (DrQ, RAD, CURL, and FERM) in their paper, and we use the same set of baselines in our replication.

Equivariant Architectures in Reinforcement Learning

A separate line of work addresses the same sample-efficiency problem by encoding symmetry directly into the network instead of learning it from data. Cohen and Welling (2016a,b) introduced group-equivariant convolutional networks, generalizing the translation-equivariant property of standard CNNs to arbitrary finite symmetry groups. Weiler and Cesa (2019) extended this to the full Euclidean group $E(2)$ through steerable convolutions, which is the framework both Wang et al. and our reimplementation build on. The first applications of equivariance to reinforcement learning were limited to small discrete symmetries in toy domains: Mondal et al. (2020) used steerable CNNs on two game environments, and van der Pol et al. (2020b) proposed MDP homomorphic networks for encoding rotational and reflectional equivariance on a small set of tasks. Wang et al. (2021) was the first to apply the idea to robotic manipulation, but their formulation was restricted to the spatial action space, where the policy outputs a target end-effector pose rather than an incremental displacement. Wang, Walters, and Platt (2022), the paper we replicate, was the first to scale architectural equivariance to close-loop manipulation with both discrete (DQN) and continuous (SAC) action spaces, and showed a large sample-efficiency advantage over the augmentation and contrastive baselines at this scale.

Extensions Since Wang et al. (2022)

The approach has since been extended along several axes. Wang et al. (2022) ported the same $SO(2)$-equivariant architecture onto a physical UR5 arm and showed that the sample-efficiency gains carry over from simulation to real hardware. Later work has generalized both the symmetry group and the policy class. Equivariant Diffusion Policy (Wang et al. 2024) combines $SO(2)$-equivariant backbones with diffusion policy heads on MimicGen. Spherical Diffusion Policy (Zhu et al. 2025) extends this to the full $SE(3)$ Euclidean group through spherical CNNs. Other work applies equivariance to related manipulation components, including grasp-pose prediction with Equivariant Descriptor Fields (Ryu et al. 2023) and 3D visuomotor policy learning from point clouds (Hu et al. 2025).

Background

Soft Actor-Critic

Soft Actor-Critic (SAC; Haarnoja et al. 2018) is the off-policy actor-critic algorithm we use for our continuous-action agents. SAC maximizes an entropy-regularized return

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\big(r_t + \alpha\,\mathcal{H}(\pi(\cdot \mid s_t))\big)\right], \qquad (1)$$

where α is an automatically tuned temperature and $\mathcal{H}$ is the policy entropy. It trains twin Q-networks Q_{ϕ_1}, Q_{ϕ_2} against the soft Bellman target

$$y = r + \gamma\left(\min_i Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right), \qquad (2)$$

a stochastic Gaussian actor π_θ that minimizes $\mathcal{L}_\pi = \mathbb{E}\big[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s,a)\big]$, and target networks $Q_{\bar{\phi}_i}$ updated by Polyak averaging at rate τ. The twin critics reduce Q-value overestimation, and the entropy bonus encourages exploration.

Deep Q-Networks

Deep Q-Networks (DQN; Mnih et al. 2015) is the off-policy value-based algorithm we use for our discrete-action agents. DQN fits a single Q-network Q_ϕ against the greedy Bellman target

$$y = r + \gamma \max_{a'} Q_{\bar{\phi}}(s', a') \qquad (3)$$

under a Huber loss, with target network $Q_{\bar{\phi}}$ updated by Polyak averaging at rate τ. Our equivariant agents inherit these SAC and DQN losses unchanged. The paper's contribution is entirely in the network architectures that realize π_θ and Q_ϕ.

Equivariant Neural Networks

A function $f\colon X \to Y$ is *equivariant* with respect to a group G acting on X and Y if $f(g \cdot x) = g \cdot f(x)$ for every $g \in G$ and $x \in X$, and *invariant* if $f(g \cdot x) = f(x)$. Invariance is a special case of equivariance in which the action on Y is trivial. For top-down robotic manipulation, the relevant group is $SO(2)$, the group of planar rotations of the workspace: rotating the world and the gripper by the same angle gives the same task. Equivariant convolutional networks approximate $SO(2)$ with its cyclic subgroup $C_n = \{2\pi k/n : k = 0, \ldots, n-1\} \subset SO(2)$, a finite group of n evenly-spaced rotations.

The action of $g \in C_n$ on a feature map depends on the *representation* chosen for the map, which specifies how the channel values change when the spatial dimensions rotate. Three representations are used in the agents we build. The *trivial representation* ρ_0 leaves channel values unchanged under rotation, which fits scalar quantities such as a depth value at a pixel. The *standard representation* ρ_1 is the natural action of C_n on $\mathbb{R}^2$: a two-channel feature v transforms as $\rho_1(g)v = R_g v$, which fits a planar vector such as the (a_x, a_y) component of an action. The *regular representation* ρ_{reg} acts on n-channel features by cyclically permuting the channels, and is the default representation inside the hidden layers of the equivariant networks.

An equivariant convolutional layer h is built so that rotating the input feature map through any $g \in C_n$ rotates the output by the same group element:

$$h(g \cdot \mathcal{F}_{\text{in}}) = g \cdot h(\mathcal{F}_{\text{in}}). \tag{4}$$

Weiler and Cesa (2019) show that this reduces to a linear condition on the convolutional kernel, and their `e2cnn` library provides these layers as drop-in replacements for ordinary convolutions. A network built entirely from equivariant layers is itself equivariant by composition, so the equivariance is enforced by the weights themselves regardless of training data. Applying this idea to reinforcement learning goes back to an older result: MDPs with group symmetries have optimal solutions with the same symmetries (Ravindran and Barto 2001; van der Pol et al. 2020b). The next section reviews the specific version of that result developed by Wang, Walters, and Platt (2022), which is what motivates the architectures of Equivariant SAC and Equivariant DQN.

Group-Invariant MDPs

Wang, Walters, and Platt (2022) formalize the connection between group symmetry and reinforcement learning through what they call a *group-invariant MDP*. Consider a discounted MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ and a group G that acts on both $\mathcal{S}$ and $\mathcal{A}$. The MDP is *G-invariant* if the transition distribution and the reward function are both preserved by the group action:

$$P(g \cdot s' \mid g \cdot s, g \cdot a) = P(s' \mid s, a), \tag{5}$$

$$R(g \cdot s, g \cdot a) = R(s, a), \tag{6}$$

for every $g \in G$. Under this assumption, Wang, Walters, and Platt (2022, Section 4) prove that the optimal Q-function is itself G-invariant and the optimal policy is G-equivariant:

$$Q^*(g \cdot s, g \cdot a) = Q^*(s, a), \tag{7}$$

$$\pi^*(g \cdot s) = g \cdot \pi^*(s). \tag{8}$$

This result is why we can build the invariance and equivariance directly into the network. The restricted class of equivariant networks still contains the optimal solution, so enforcing the constraint does not exclude any valid policy or Q-function. It also substantially reduces the parameter count and the amount of training data needed to recover the solution.

The close-loop manipulation tasks we use satisfy the C_n-invariance condition because of their top-down workspace geometry. The observation is a depth image rendered from a camera that tracks the gripper, and the depth value at each pixel is a scalar quantity that does not change when the workspace rotates, so the state transforms through the trivial representation ρ_0 at every pixel. The action decomposes into a planar displacement a_{xy} that transforms under the standard representation ρ_1 and into the remaining components $(a_\lambda, a_z, a_\theta)$ (gripper open/close, vertical displacement, and rotation about the vertical axis), which are scalars under any rotation of the workspace and therefore transform under ρ_0. The reward returns $+1$ when the agent reaches the goal and 0 otherwise, and this criterion does not depend on the orientation of the workspace. These three conditions make each task a C_n-invariant MDP and motivate the actor and Q-network architectures we describe next.

Methods

Under the Group-Invariant MDP framework, we can use a Q-function that is C_n-invariant and a policy that is C_n-equivariant with no loss of optimality. Equivariant DQN and Equivariant SAC apply this constraint by replacing the convolutional backbone of the non-equivariant baselines with an $E(2)$-equivariant network from `e2cnn` (Weiler and Cesa 2019). In both cases the input is a two-channel 128×128 trivial-representation (ρ_0) feature map (a top-down depth heightmap and a binary gripper-holding channel), and the symmetry group is the cyclic subgroup $C_n \subset SO(2)$. The loss structures and optimization schedules from Section 3 are used unchanged. The network architecture is the paper's entire methodological contribution.

Equivariant DQN

Equivariant DQN operates on the factored discrete action space $\mathcal{A} = A_\lambda \times A_{xy} \times A_z \times A_\theta$, where A_{xy} is a 3×3 grid of planar end-effector displacements and A_λ, A_z, A_θ are small discrete sets for the gripper, vertical displacement, and rotation about the vertical axis. Wang, Walters, and Platt (2022) use the group C_4 and a seven-layer steerable CNN Q_ϕ whose output is an $|A_\lambda| \cdot |A_z| \cdot |A_\theta|$-channel, 3×3-spatial trivial-representation feature map. The output layout uses both factorizations that C_4-equivariance allows: each spatial cell of the 3×3 grid corresponds to one choice of a_{xy}, and the channel axis indexes every combination of the remaining invariant action components. Rotating the input depth image by 90° permutes the 3×3 spatial cells by the same rotation, so that $Q^*(g \cdot s, g \cdot a_{xy}, a_{\text{inv}}) = Q^*(s, a_{xy}, a_{\text{inv}})$ after the spatial permutation. The channel values do not change under the rotation, which gives $Q^*(g \cdot s, a_{xy}, a_{\text{inv}}) = Q^*(s, a_{xy}, a_{\text{inv}})$ for the invariant action components. Action selection picks the $(a_{xy}, a_\lambda, a_z, a_\theta)$ with the largest predicted value across the full output volume. We reimplement the same architecture with the same C_4 group and train it with the DQN loss from Section 3.2 on the paper's close-loop manipulation tasks.

Equivariant SAC

Equivariant SAC is continuous-action, so the symmetry is encoded directly into the actor and critic rather than into a gridded output. Wang, Walters, and Platt (2022) use C_8 (eight evenly spaced rotations, finer than DQN's C_4) and build an eight-layer equivariant actor π_θ together with an invariant twin critic (Q_{ϕ_1}, Q_{ϕ_2}).

The actor outputs a mean action $\bar{a} = (a_{xy}, a_{\text{inv}}, \sigma)$. Here $a_{xy} \in \mathbb{R}^2$ is the planar end-effector displacement, which transforms under the standard representation ρ_1. The pair (a_{inv}, σ) contains the invariant action components (gripper open/close, vertical displacement, rotation about the vertical axis) together with the policy's per-dimension log-standard-deviation, both transforming under the trivial representation ρ_0. The final layer of the actor emits one ρ_1 output and several ρ_0 outputs, which makes $\pi_\theta(g \cdot s) = g \cdot \pi_\theta(s)$ hold by construction.

The critic $Q_\phi(s, a)$ has to be C_n-invariant jointly in its state and action arguments. Wang et al. build it by first en-

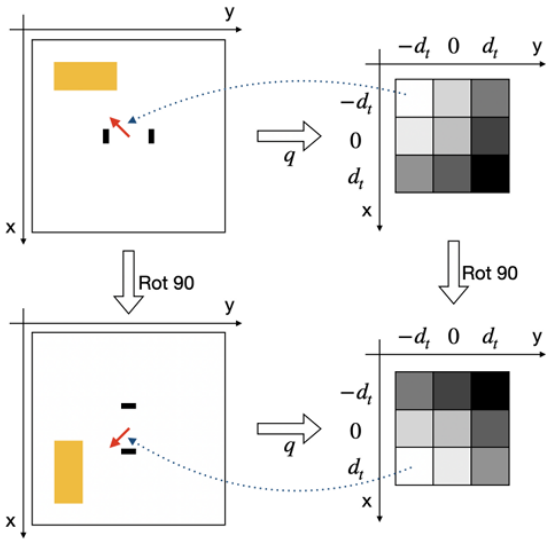


Figure 1: Equivariant DQN Q-function. The input depth map (left) is passed through a steerable CNN q to produce a 3×3 spatial output where each cell scores one planar action a_{xy}; rotating the input by $90°$ permutes the cells by the same rotation so Q^* is C_4-invariant jointly in (s, a_{xy}). Adapted from Wang, Walters, and Platt (2022).

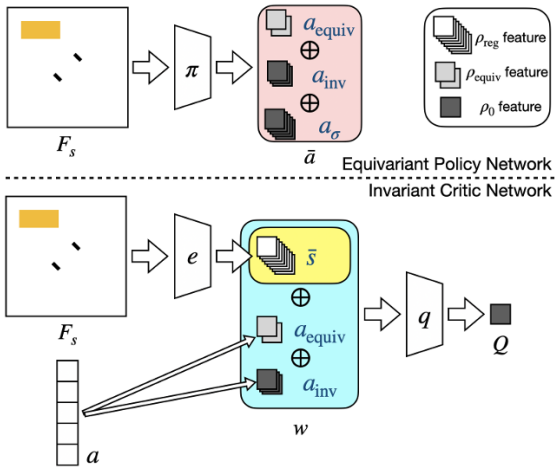


Figure 2: Equivariant SAC architecture. The actor π emits one ρ_1-typed output a_{equiv} (planar end-effector displacement), several ρ_0-typed outputs a_{inv} (gripper, vertical displacement, rotation about the vertical axis), and the per-dimension log-standard-deviations a_σ. The critic q encodes the state with an equivariant CNN e to a regular-representation feature $\bar{s}$, concatenates it with representation-tagged action slots, and reduces to a single invariant scalar Q. Adapted from Wang, Walters, and Platt (2022).

coding the state with an equivariant CNN whose output is a regular-representation (ρ_{reg}) feature vector, concatenating this with a representation-tagged copy of the action (the ρ_1 component of the action fills a ρ_1 slot, the invariant components fill ρ_0 slots), and then passing the result through equivariant layers that reduce to a single invariant scalar. Collapsing the regular representation to the trivial representation needs a non-linear operation, specifically a group-pooling $\max$ across the n cyclic channel copies, rather than a linear projection. The paper shows in its Appendix B that a linear collapse would over-constrain the Q-function to the form $Q(s, a) = \sum_i v_i(s, a)$ and silently lose representational power. The non-linear group-pooling avoids this. The twin critics share the same architecture and are Polyak-averaged to their target copies as in standard SAC.

We match the paper's C_8 group, the $\rho_0/\rho_1/\rho_{\text{reg}}$ decomposition at the action and feature-map interfaces, and the non-linear group-pooling in the critic. The SAC losses and optimization schedule from Section 3.1 are used unchanged, with the paper's hyperparameters (Appendix F of Wang, Walters, and Platt 2022).

Experiments

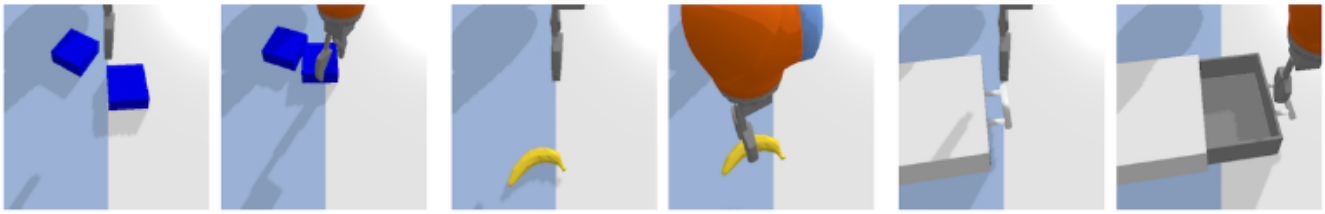
Experimental Setup

All experiments follow Wang, Walters, and Platt (2022)'s close-loop protocol: 20,000 training steps, five-episode evaluation every 500 environment steps, and a replay buffer pre-loaded with 20 expert episodes from a hand-coded planner. Hyperparameters are taken from Appendix F of the paper for SAC and Table 8 (Section 6.1) for DQN; in particular, SAC uses $\gamma = 0.99$, $\tau = 0.01$, and initial temperature $\alpha_0 = 0.01$, while DQN uses $\gamma = 0.95$, batch size 32, and step sizes $|A_{xy}| = 0.05\,\text{m}$, $|A_z| = 0.02\,\text{m}$, $|A_\theta| = \pi/16$. We report the mean discounted evaluation return with ± 1 standard-deviation bands over two random seeds. Each training run uses five parallel PyBullet environments for collection on a laptop-class NVIDIA RTX 3000 Ada GPU (8 GB VRAM).

Equivariant SAC on BulletArm

Equivariant SAC was evaluated against CNN SAC, DrQ SAC, FERM SAC, and RAD SAC on three BulletArm tasks: block pulling, block picking, and drawer opening. Block picking was used in place of the paper's object-picking task because our SAC sweep was locked in early and no household-object SAC runs were completed before the scope freeze. Block picking is the sparse-reward cube variant of the same grasp-and-lift problem.

Learning curves are shown in Figure 4. Equivariant SAC reached a discounted evaluation reward of approximately 0.93 on all three tasks within about 2,500 training steps and held that plateau for the remaining 17,500 steps. The early convergence is a product of the expert buffer, the C_8 equivariance constraint, and an online $SO(2)$ replay augmentation, each of which contributes sample-efficiency on its own. The 20 pre-loaded expert episodes give the agent a starting distribution of successful trajectories, the equivariance constraint implicitly multiplies each of those episodes by the 8 rotations in the group, and the online $SO(2)$ buffer adds four rotated copies per transition collected during training. The non-equivariant baselines did not learn block picking or



(a) Block Pulling (b) Object Picking (c) Drawer Opening

Figure 3: The three BulletArm close-loop manipulation tasks used throughout our PyBullet experiments: block pulling, block picking (SAC) / object picking (DQN), and drawer opening. Each rollout is initialized with a random object pose and a gripper-centered top-down depth camera is used as the observation.

drawer opening and stayed within $[0, 0.05]$ across all $20{,}000$ steps. DrQ partially learned block pulling and reached approximately 0.47 by the end of training, consistent with the paper's result. CNN, RAD, and FERM stayed near zero on that task. Our FERM implementation is partially crippled by the omission of the $1{,}600$-step contrastive pre-training phase used by Zhan et al. (2020); the InfoNCE loss is instead applied concurrently with SAC training.

Equivariant DQN on BulletArm

Equivariant DQN was evaluated against CNN DQN, DrQ DQN, CURL DQN, and RAD DQN on block pulling, object picking (BulletArm's household-object grasp-and-lift task, i.e. Wang et al. Figure 6(b)), and drawer opening. Results are shown in Figure 5. On block pulling, Equivariant DQN led at convergence with a discounted eval return of approximately 0.20, compared to 0.05–0.10 for CNN, DrQ, and CURL. On object picking, Equivariant DQN reached approximately 0.50, ahead of the data-augmentation baselines (0.30–0.40), which themselves were well above random. On drawer opening, where time and compute budget allowed only a single seed per variant, Equivariant DQN and CNN DQN were statistically indistinguishable and both reached approximately 0.45. The equivariance advantage on DQN was visible but smaller than on SAC at this step budget, matching the trend in the paper's Figure 6: Equivariant DQN outperforms its baselines by a narrower margin than Equivariant SAC does.

Extension to ManiSkill 3

The SAC pipeline was ported from BulletArm to ManiSkill 3 (Tao et al. 2024), a GPU-parallel manipulation simulator built on SAPIEN, as a novel extension to the paper. Both IsaacLab and ManiSkill were considered. ManiSkill 3 was chosen because its image-observation pipeline is better documented and because its GPU-batched simulation replaces the CPU multi-processing used in the paper's BulletArm sweeps, which raises single-process throughput from roughly 4 SAC steps per second on PyBullet to roughly 42 SAC steps per second on ManiSkill.

The observation and action spaces were chosen to replicate those of the paper. ManiSkill 3's default sensors did not include an overhead, gripper-aligned camera, so a custom camera was registered via `sensor_configs` that places a top-down 128×128 depth camera above the workspace. PickCube-v1 was subclassed into `EquiPickCube-v1` to render the robot arm invisible in the depth stream, which removes the non-equivariant arm geometry that would otherwise be visible in the observation. The `pd_ee_delta_pose` control mode was used so that the action space is a relative end-effector displacement, matching the paper's factored action space. Training ran with 32 GPU-batched environments per process rather than the 5 CPU processes used on BulletArm.

Equivariant SAC and CNN SAC were then run on PickCube-v1, the ManiSkill 3 task closest to the paper's object-picking setup, with four seeds each for $100{,}000$ training steps under ManiSkill's default shaped reward. Results are shown in Figure 6.

Equivariant SAC reaches a higher mean return than CNN SAC throughout training and continues pulling ahead as the budget grows. By $100{,}000$ steps, Equivariant SAC is at roughly 8 mean return compared to roughly 5 for CNN SAC, and the gap is visible as early as $15{,}000$ steps. This is the same direction as the BulletArm result, where equivariance produces a large sample-efficiency advantage over the CNN baseline, and shows that the advantage survives a change of simulator, physics backend, and reward function.

Challenges

Porting the equivariant pipeline from BulletArm to a GPU-parallel simulator was not a smooth process. The BulletArm environments used in the original paper were custom-built for equivariance and expose an observation that no existing manipulation simulator provides by default: a depth image rendered from a virtual camera that tracks the gripper's horizontal position, points straight down, and covers a fixed physical area of the workspace regardless of gripper height, with the robot arm hidden from the render. This rig is not a reasonable physical camera and was not a convenience option in ManiSkill. It was approximated in ManiSkill by registering a top-down overhead camera through the simulator's sensor configuration interface, and by subclassing PickCube-v1 to render the robot arm invisible so that the arm geometry does not enter the depth image. The depth values were then clipped and centered so that their range

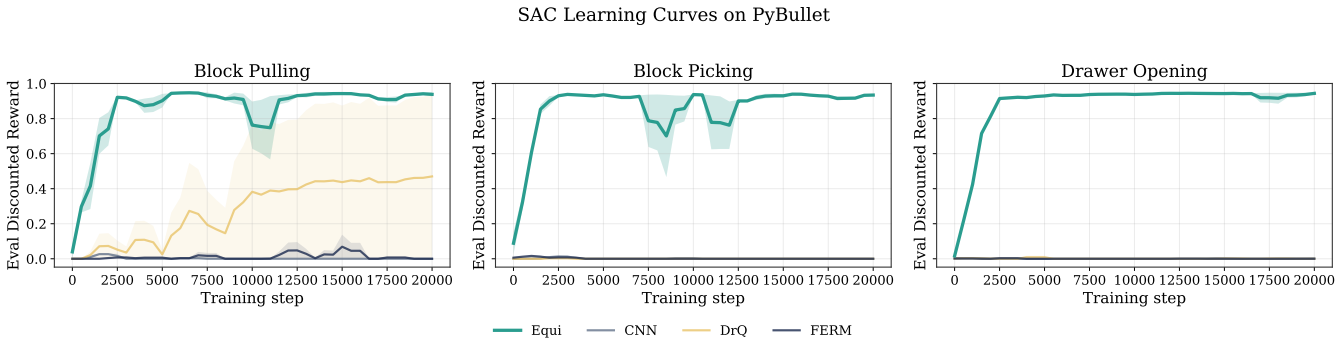


Figure 4: Comparison of Equivariant SAC (teal) with the CNN, DrQ, and FERM baselines on the close-loop BulletArm tasks. The plots show the evaluation performance of the greedy policy in terms of the discounted reward. Evaluation is performed every 500 training steps. Results are averaged over two seeds; shading denotes standard deviation. Curves are smoothed with a three-point rolling mean.

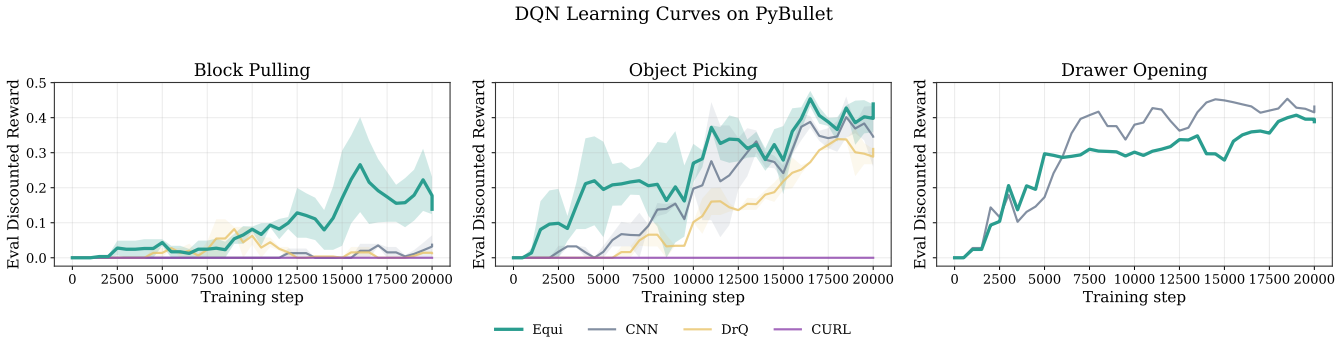


Figure 5: Comparison of Equivariant DQN (teal) with the CNN, DrQ, and CURL baselines on the close-loop BulletArm tasks. The plots show the evaluation performance of the greedy policy in terms of the discounted reward. Evaluation is performed every 500 training steps. Results are averaged over two seeds for block pulling and object picking, and a single seed for drawer opening; shading denotes standard deviation. Curves are smoothed with a three-point rolling mean.

Figure 6: Comparison of Equivariant SAC (blue) with the CNN SAC baseline on ManiSkill 3 PickCube-v1 under the default shaped reward. The plot shows the mean episode return at evaluation time. Results are averaged over four seeds; shading denotes standard deviation.

matches the BulletArm observation. This observation-space mismatch was the biggest source of debugging time in the ManiSkill port.

Compute was the other hard constraint. The experiments in this report were run on a single laptop-class 8 GB GPU, with a small amount of additional throughput from a shared lab machine. Runs on the lab machine were kept conservative so that they would not exhaust the GPU memory shared with another user, and the training budget was limited to 20,000 steps per run at two seeds per variant rather than the four seeds used by the paper. The shared lab machine also uses an NVIDIA Blackwell-generation GPU, and the SAPIEN physics binaries shipped with our ManiSkill install do not yet support that architecture. A debug pass was attempted and was deferred so that the shared environment would not be destabilized.

A few small details cost time to track down. DQN's xy step size has to match between the expert planner and the discrete action grid used at decode time. An off-by-0.03 m between the two silently corrupts the replay buffer, since the stored grid index no longer matches the physical motion in the simulator. The paper's augmentation baselines use shift for DrQ and crop for RAD, CURL, and FERM at

specific pixel budgets, not the $SO(2)$ rotation augmentation we started with. The BulletArm single-runner also needs an explicit auto-reset flag between evaluation episodes, which is not set by default. Without it, every eval episode after the first runs a single step and the eval statistics collapse to a uniform episode length of 10.8. All three were caught in the two days before submission.

With more time, the project could have been extended in several directions. A larger compute budget would have allowed four seeds per variant on the BulletArm sweeps, matching the paper's evaluation protocol. The FERM SAC baseline would have been re-run with the $1{,}600$-step contrastive pre-training phase described in Zhan et al. (2020), rather than the concurrent InfoNCE loss used here. The ManiSkill extension would have covered the remaining augmentation and contrastive baselines (DrQ, RAD, FERM) alongside CNN SAC, and would also have been scaled to block pulling and drawer opening so that the BulletArm-to-ManiSkill comparison covers the full task set. Resolving the SAPIEN-on-Blackwell kernel incompatibility would have also unlocked the shared lab machine for that larger budget.

Conclusion

The main result of Wang, Walters, and Platt (2022) was reproduced on three BulletArm close-loop manipulation tasks. Under sparse reward and a small number of expert demonstrations, equivariance was the difference between learning and not learning for SAC. Equivariance was also an advantage for DQN, although smaller and slower to emerge. The paper's architectural claim, that enforcing C_n-equivariance in the policy and C_n-invariance in the Q-function is both sufficient and far more sample-efficient than learning the same symmetry from data, held up in our reimplementation at two seeds per variant.

The ManiSkill 3 extension also worked. Equivariant SAC and CNN SAC now run on a GPU-parallel simulator with a gripper-aligned depth observation, scripted expert demonstrations, prioritization, and online $SO(2)$ buffer augmentation, all ported from the BulletArm pipeline. Equivariant SAC outperformed CNN SAC on PickCube-v1 across four seeds and $100{,}000$ training steps, which shows that the equivariance advantage carries over from BulletArm to ManiSkill even under a shaped-reward setting and a different physics backend.

Our reimplementation of the paper's BulletArm sweeps and the ManiSkill 3 extension is available at https://github.com/wade-g-williams/so2_equi_rl. Joseph Horwitz's ManiSkill 3 SAC implementation, on which the extension builds, is available at https://github.com/joeyhz/eq_sac.

References

Cohen, T. S.; and Welling, M. 2016a. Group Equivariant Convolutional Networks. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*.

Cohen, T. S.; and Welling, M. 2016b. Steerable CNNs. In *International Conference on Learning Representations (ICLR)*. ArXiv:1612.08498.

Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 1861–1870.

Hu, B.; Wang, D.; Klee, D.; Tian, H.; Zhu, X.; Huang, H.; Platt, R.; and Walters, R. 2025. 3D Equivariant Visuomotor Policy Learning via Spherical Projection. *arXiv preprint arXiv:2505.16969*.

Kostrikov, I.; Yarats, D.; and Fergus, R. 2021. Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels. In *International Conference on Learning Representations (ICLR)*.

Laskin, M.; Lee, K.; Stooke, A.; Pinto, L.; Abbeel, P.; and Srinivas, A. 2020. Reinforcement Learning with Augmented Data. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Laskin, M.; Srinivas, A.; and Abbeel, P. 2020. CURL: Contrastive Unsupervised Representations for Reinforcement Learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Mondal, A. K.; Jain, P.; Siddiqi, K.; and Ravanbakhsh, S. 2020. Group Equivariant Deep Reinforcement Learning. *arXiv preprint arXiv:2007.03437*.

Ravindran, B.; and Barto, A. G. 2001. Symmetries and Model Minimization in Markov Decision Processes. Technical Report UM-CS-2001-043, University of Massachusetts Amherst.

Ryu, H.; Lee, H.-i.; Lee, J.-H.; and Choi, J. 2023. Equivariant Descriptor Fields: SE(3)-Equivariant Energy-Based Models for End-to-End Visual Robotic Manipulation Learning. In *International Conference on Learning Representations (ICLR)*.

Tao, S.; Xiang, F.; Shukla, A.; Qin, Y.; Hinrichsen, X.; Yuan, X.; Bao, C.; Lin, X.; Liu, Y.; Chan, T.-k.; Gao, Y.; Li, X.; Mu, T.; Xiao, N.; Gurha, A.; Huang, Z.; Calandra, R.; Chen, R.; Luo, S.; and Su, H. 2024. ManiSkill3: GPU Parallelized Robotics Simulation and Rendering for Generalizable Embodied AI. *arXiv preprint arXiv:2410.00425*.

van den Oord, A.; Li, Y.; and Vinyals, O. 2018. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748*.

van der Pol, E.; Kipf, T.; Oliehoek, F. A.; and Welling, M. 2020a. Plannable Approximations to MDP Homomorphisms: Equivariance under Actions. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*.

van der Pol, E.; Worrall, D.; van Hoof, H.; Oliehoek, F. A.; and Welling, M. 2020b. MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wang, D.; Hart, S.; Surovik, D.; Kelestemur, T.; Huang, H.; Zhao, H.; Yeatman, M.; Wang, J.; Walters, R.; and Platt, R. 2024. Equivariant Diffusion Policy. In *Conference on Robot Learning (CoRL)*.

Wang, D.; Jia, M.; Zhu, X.; Walters, R.; and Platt, R. 2022. On-Robot Learning with Equivariant Models. In *Conference on Robot Learning (CoRL)*.

Wang, D.; Walters, R.; and Platt, R. 2022. SO(2)-Equivariant Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Wang, D.; Walters, R.; Zhu, X.; and Platt, R. 2021. Equivariant Q Learning in Spatial Action Spaces. In *Conference on Robot Learning (CoRL)*.

Wang, R.; Walters, R.; and Yu, R. 2020. Incorporating Symmetry into Deep Dynamics Models for Improved Generalization. In *International Conference on Learning Representations (ICLR)*.

Weiler, M.; and Cesa, G. 2019. General E(2)-Equivariant Steerable CNNs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zhan, A.; Zhao, P.; Pinto, L.; Abbeel, P.; and Laskin, M. 2020. A Framework for Efficient Robotic Manipulation. *arXiv preprint arXiv:2012.07975*.

Zhu, X.; Wang, F.; Walters, R.; and Shi, J. 2025. SE(3)-Equivariant Diffusion Policy in Spherical Fourier Space. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. ArXiv:2507.01723.